Python Engineering - Functions

validate_inputs(), booleanos y diferencia entre definir y llamar una funcion

Nota de apoyo de **week_01_logic -> class_04_functions**. Este documento resume el bug encontrado en EX12 y por que la correccion con **True / False** es una forma natural de representar una validacion.

1. Una funcion tambien es un objeto

Cuando Python lee una definicion con **def**, crea un objeto funcion y lo guarda bajo un nombre. El cuerpo no se ejecuta en ese momento.

```
def validate_inputs(h0, v0, t0, g, dt):
    ...

# Aqui NO ejecutamos la funcion:
validate_inputs

# Aqui SI la ejecutamos:
validate_inputs(h0, v0, t0, g, dt)
```

Idea clave: `validate_inputs` es el objeto funcion. `validate_inputs(...)` es una llamada que ejecuta el cuerpo y devuelve un resultado.

2. Por que el primer if no funcionaba

```
if validate_inputs is None:
    print('Invalid inputs')
```

Esa condicion pregunta si el *objeto funcion* es None. No llama a la funcion, por lo que nunca se validan h0, v0, t0, g ni dt.

3. El segundo problema: None para valido e invalido

En la primera version se devolvia **None** cuando un dato era invalido, pero si todos los datos eran validos la funcion llegaba al final sin un return. En Python, llegar al final de una funcion equivale tambien a devolver None.

Caso	Resultado de la primera version
Input invalido	None
Input valido	None

4. La solucion natural: devolver un booleano

```
def validate_inputs(h0, v0, t0, g, dt):
    if h0 < 0:
        return False
    if v0 <= 0:
        return False
    if t0 < 0:
        return False
    if dt <= 0:
        return False
    if g <= 0:
        return False
    return True
```

Ahora la funcion responde una pregunta binaria: **son validos los inputs?** Por eso True/False expresa la intencion mejor que None.

```
if validate_inputs(h0, v0, t0, g, dt):
    # Run simulation
    ...
```

Se puede leer casi como ingles: **if validate inputs, run the simulation**.

5. Que ocurre durante la llamada

Al ejecutar la llamada, Python enlaza los argumentos con los parametros, crea el scope local de esa llamada, ejecuta las comprobaciones en orden y termina inmediatamente cuando encuentra un return.

```
CALL
  -> bind arguments
  -> create local scope
  -> evaluate validations
  -> return False OR True
  -> continue in main
```

6. Guard clauses

Como cada caso invalido hace return False, no necesitamos elif. Si Python llega al siguiente if, sabemos que el anterior no termino la funcion.

Patron: rechazar pronto los casos invalidos y dejar el camino principal limpio. A este estilo se le suele llamar *guard clauses*.

7. Conexion con EX12

En EX12 el flujo queda conceptualmente: validar -> simular -> solucion analitica -> errores -> mostrar. La validacion ya no es solo un print: controla si el resto del programa puede ejecutarse.

Learning tracker

Concepto	Estado
Objeto funcion vs llamada	Aprendido
Return implicito None	Aprendido
Boolean return	Aprendido
Guard clauses	Introducido
Control del flujo desde main	Aprendido